

# ByteBlower algorithms and mechanisms

Every algorithm, formula and decision rule the ByteBlower docs describe, written out. Where the docs name something without explaining it, that is said at the end.

## Rate arithmetic

A frame-blasting stream is defined by three numbers: the frame, the gap between frame starts in nanoseconds, and how many frames to send. Everything else is derived.

On the wire each frame costs 24 bytes more than the frame you define: preamble 7, start delimiter 1, checksum 4, inter-frame pause 12. With frame size  $S$  in bytes (no checksum) and a target load  $R$  in bits per second:

$$\text{frames per second} = \frac{R}{8(S + 24)} \quad \text{gap}_{\text{ns}} = \left\lceil \frac{10^9}{\text{frames per second}} \right\rceil$$

The gap is rounded up because the API takes whole nanoseconds. Test duration is gap times number of frames.

Worked example at line rate, 1 Gbit/s with 60-byte frames: 84 bytes per frame on the wire gives 1,488,095 frames per second, which is only 714 Mbit/s if you count the frames alone ( $60/84 = 71.4\%$ ).

| Bitrate convention | Bytes counted per frame |
|--------------------|-------------------------|
| Frame only         | $S$                     |
| Frame and checksum | $S + 4$                 |
| Physical           | $S + 24$                |

The GUI lets you choose the convention; the Test Framework names them `frame`, `frame_with_fcs` and `physical`, and its `bitrate` setting excludes VLAN tag bytes. Comparing two results that use different conventions is the most common arithmetic mistake.

Other derived rules:

- A flow aimed at a **port group** has its rate divided evenly over the members: 3 Gbit/s to a group of four sends 750 Mbit/s to each.
- The port's MTU caps the layer-3 packet. Frames larger than it cannot be sent; TCP sizes its segments from it. It does not limit received frames.

- A VLAN tag adds 4 bytes per tag to every frame, added on send and required on receive.

## Counting and loss

The server has no concept of a flow. A flow is a transmit counter on one port paired with a receive filter on another, and loss is the difference.

1. The sending stream counts every frame it transmits.
2. On the destination port a **trigger** counts every received frame matching a BPF filter. The GUI and Test Framework generate a filter unique to the flow, typically source and destination address, UDP ports and frame length.
3. Triggers count from the moment they exist, so counters are cleared just before traffic starts.
4. After the stream ends, wait about a second so frames still in flight are counted.
5. Loss follows RFC 1242:

```
\text{loss}\,, \% = 100 \times \frac{\text{sent} - \text{received}}{\text{sent}}
```

With several flows, the aggregate loss sums sent and received over all flows that have a receiver. Flows with no receiver are background load and are left out.

**Collision detection.** Before a run, the GUI checks that every flow's filter is unique. If two flows have identical or near-identical frames, no unique filter exists, frames are counted twice, and loss goes negative. It warns at start and notes it in the report. Fixes: a per-frame unique modifier on layer 4, a different length, one changed payload byte, or different UDP ports.

**Scouting.** Before the measured traffic, a single scouting frame per stream can be sent so switches and gateways learn addresses (ARP and forwarding entries). The docs give the purpose as initialising network entries such as ARP and verifying the connection. It is not used for traffic sent from an Endpoint.

**Locating loss in time.** The docs give a bisection procedure:

1. Read the over-time graph and zoom in on the dip.
2. Shorten the test. If loss % falls, there are several loss events; if it stays the same, loss is continuous; if it rises, the main event is at the start.
3. Split one long flow into many short back-to-back flows, for example 10 seconds into 20 flows of 500 ms, and read which window lost frames.
4. To identify the exact frames, enable sequence numbers, capture, and read the numbers.

## Latency, jitter and reordering

Both measurements work by writing a tag into the frame payload at send time and reading it at the receiver.

### Latency

1. Just before a frame leaves, the sender overwrites 8 bytes at the end of the payload with its local time.
2. The receiving port reads the tag and subtracts it from its own local time. That difference is one-way latency.
3. Per sampling interval and cumulatively it keeps minimum, average, maximum, jitter, and counts of valid and invalid packets.

The tag has three formats: microseconds, microseconds with a check code, and ten-nanosecond units. The receiver runs a sanity check on each tag when the format allows it. A frame that matches the filter but fails the check is still counted as received but gives no latency value, so the received count can exceed the count used for latency.

How corruption shows up:

| What happened to the frame                            | Result                                 |
|-------------------------------------------------------|----------------------------------------|
| Ethernet checksum fails                               | Dropped by the network; counts as loss |
| Checksum valid, timestamp off by more than one minute | Counted as invalid; no latency value   |
| Checksum valid, timestamp off by less than one minute | A latency spike of up to seconds       |

The tag changes the payload, so it interacts with the UDP checksum. Server models with hardware timestamping could not always recompute it, which produced bad checksums and up to 100% loss on IPv6 (where a zero checksum is not allowed). Current versions default to software timestamping, which corrects the checksum. Because the tag overwrites the end of the frame, two flows must differ somewhere other than their last 8 bytes.

Because the result is a difference of two clocks, both ports on one server share a clock and are exact. Across machines the clocks must be disciplined continuously, or the offset drifts into the result and can make it negative.

## Latency distribution and percentiles

The distribution receiver sorts each packet's latency into equal-width buckets over a range, default 0 to 1 second, with one extra bucket below the range and one above. Setting the range clears the counters.

The report turns the buckets into a cumulative curve. For a latency  $x$ ,  $CDF(x)$  is the fraction of packets at or below  $x$  and  $CCDF(x) = 1 - CDF(x)$ . Percentile P99 is the  $x$  where the CDF reaches 0.99. Both axes are logarithmic and the plot ends at P99.99.

The quality of that curve depends on the buckets, which is what the report warnings test:

| Condition                          | Meaning                                                                  |
|------------------------------------|--------------------------------------------------------------------------|
| Fewer than 50 occupied data points | Range too wide for the data; all packets sit in a few buckets            |
| Packets below or above the range   | The tail is cut off; percentiles there are unknown                       |
| Fewer than 10,000 received packets | Too few samples for a stable P99.99                                      |
| Sampled packets                    | A 100 Gbit/s system measured only a sample; rare outliers may be missing |

The Test Framework's CDF analyser sets its range to 0 up to 50 times the latency threshold, then tests the latency at the configured quantile (default 99.9) against the threshold (default 5 ms).

## Reordering

1. The sender writes an incrementing sequence number into an 8-byte field at the end of the payload: the first 2 bytes are used for checksum correction, the last 6 are the frame number.
2. The receiver tracks the numbers and counts packets that arrive out of sequence, plus valid and invalid tags and the biggest gap seen.
3. When a frame carries both tags, the sequence field moves ahead of the timestamp; position 16 from the end is the recommended placement.

It reports reordering, not which frames went missing. To find those, capture and read the 6-byte numbers.

The docs note that random-size frames can cause high out-of-sequence counts, without giving the reason.

## Throughput search

Throughput is not measured directly; it is searched for by running loss trials at different rates.

### RFC 2544 search (Test Framework)

Per frame size, with a tolerated loss, a wanted accuracy and an iteration limit ([docs](#)):

```
rate = initial_bitrate                # e.g. the link's theoretical
maximum
run a trial at rate, measure loss

# Phase 1: find a failing and a passing rate
if loss > tolerated:
    repeat: rate = rate / 2            until a trial passes
else:
    repeat: rate = rate * 2 (or * 1.5) until a trial fails

# Phase 2: bisection
repeat:
    step = 50% of |difference between the last two rates|
    if last trial passed: rate = rate + step
    else:                 rate = rate - step
    run a trial
until (step is within accuracy AND last trial passed) OR iterations ==
max_iterations

result = last rate the device handled within tolerated loss
```

A trial passes when its frame loss is at or below `tolerated_frame_loss`. Each step halves the uncertainty, so reaching accuracy  $a$  from a starting gap  $g$  takes about  $\log_2(g/a)$  trials.

Frame sizes come from RFC 2544: 64, 128, 256, 512, 1024, 1280 and 1518 bytes with checksum, configured as 60 to 1514 without it. The RFC asks for at least five sizes and trials of at least 60 seconds.

Around the search:

- Before running, inputs are validated: missing parameters, address and gateway in different subnets, server reachable, interface exists.
- Runtime errors (duplicate frames, all frames lost, a layer-3 mismatch) trigger automatic reruns. Recovered errors only raise an error count; exceeding the allowed reruns fails the test.
- Verdict: FAIL if any frame size's result is below its `expected_bitrate`, or an error stopped the run.

## Manual sweep (GUI)

The throughput wizard is a linear scan, not a search: it creates one flow per load from a start speed to a final speed in fixed increments, all in one scenario, and you read where loss begins.

The documented way to use it is two passes: a coarse batch with widely spaced rates (for example 100 Mbit/s to 1 Gbit/s in 100 Mbit/s steps), then a fine batch around the limit found (for example 10 Mbit/s steps). Latency is then measured at the highest lossless rate.

The GUI also has an RFC 2544 wizard taking frame sizes, the duration of one iteration and the acceptable loss; total run time depends only on the first two.

## NAT discovery

A NAT device rewrites the source address and usually the source port, so the receiver cannot filter on the addresses the sender used. ByteBlower learns the translation before the test ([article](#)).

### Upstream: the sender is behind NAT

1. Build a discovery frame from the real test frame: same MAC addresses, VLAN tags, IP addresses, layer-4 protocol and ports. Replace the payload with a short text plus a unique token. The frame is about 100 bytes.
2. Send it from the port behind NAT at about 10 frames per second (about 8 kbit/s) for at most 20 seconds.
3. On the public side, capture with a filter on the destination address and port, and compare each payload with the token.
4. The source address and source port of the matching frame are the public mapping. The receive filter for the real flow is built from them.

### Downstream: the receiver is behind NAT

1. Run the upstream discovery from the destination port. This opens a mapping in the NAT device.
2. Rewrite the test frame's destination IP address and UDP port to the public mapping learned.
3. Send. The NAT device forwards the traffic through the mapping.

Results are reused by flows with the same frames and ports. Most NAT devices keep a mapping at least 2 minutes; if a flow starts late, the mapping can expire and a new, different one gives 100% loss. The keep-alive option sends traffic from the inside port to

hold the mapping open. Since 2.11.4 the GUI keeps mappings alive until the test starts.

The Tcl helper does the same with 256-byte packets and returns the public IP and port. For IPv6 there is no translation, but a firewall still blocks inbound traffic; the same inside-out discovery opens it.

## Direction rule for TCP

A connection can only be opened from inside the NAT. So the HTTP method is chosen from where the data source sits:

| Data source    | Method | Who opens the connection                                      |
|----------------|--------|---------------------------------------------------------------|
| Not behind NAT | GET    | The destination (client) connects to the source and downloads |
| Behind NAT     | PUT    | The source (client) connects out and uploads                  |

AUTO applies exactly this rule.

## TCP

ByteBlower runs its own TCP stack on each port, so the congestion behaviour is a setting you choose, not whatever an operating system does.

## Congestion control

Slow start and exponential backoff are always on. Below the slow-start threshold the window grows exponentially; above it, linearly. The selectable part is loss recovery ([article](#)):

| Option                               | Behaviour                                                                                                                            |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| None                                 | Fast retransmit only, no fast recovery                                                                                               |
| New Reno                             | Fast recovery that handles several dropped packets in one window                                                                     |
| SACK                                 | The receiver reports exactly which segments are missing. Negotiated at connection setup; falls back to New Reno if the peer lacks it |
| New Reno with Cubic, SACK with Cubic | Adds Cubic window growth, which recovers better on high-latency paths                                                                |

| Option           | Behaviour                                                                                     |
|------------------|-----------------------------------------------------------------------------------------------|
| TCP Prague (L4S) | Reacts to ECN congestion marks instead of waiting for loss; the count of CE marks is reported |

The docs call SACK with Cubic the best performer in most situations.

## Window sizing

The receive window bounds how much data can be in flight. It is an unscaled value of at most 65535 bytes multiplied by a power of two:

$$\text{window} = \text{initial window} \times 2^{\text{scale}} \quad \text{target: window} \geq \text{BDP} = \text{RTT} \times \text{bandwidth}$$

The window should be at least the bandwidth-delay product and a multiple of the MTU. Worked example: 1 Gbit/s at 10 ms round trip is 1.25 MB in flight;  $65535 \times 2^4$  is 1.05 MB, too small, so the scale must be 5 or more.

The diagnosis rules follow directly:

| Graph shows                                        | Rule                                                    | Cause                        |
|----------------------------------------------------|---------------------------------------------------------|------------------------------|
| Slow climb to full speed                           | Threshold reached too early, growth turned linear       | Slow-start threshold too low |
| Flat speed below the link rate, no retransmissions | Window smaller than the BDP                             | Scale too low                |
| Full speed, round-trip time above 100 ms           | Window far above the BDP; data queues inside the sender | Scale too high               |
| Sawtooth speed, retransmissions                    | Loss is collapsing the window                           | Packet loss on the path      |
| Round-trip time rising steadily                    | Queues filling                                          | Buffering in the network     |

## Timing points and the throughput formula

A TCP flow is one HTTP request. Each side records three timestamps, T1 to T3, whose meaning depends on the method ([article](#)).

|                | T1                                | T2                                   | T3                                    |
|----------------|-----------------------------------|--------------------------------------|---------------------------------------|
| GET,<br>client | Request sent                      | First response segment<br>received   | Last response segment<br>received     |
| GET,<br>server | Request received                  | Response starts                      | Last response segment<br>acknowledged |
| PUT,<br>client | Request starts                    | Last request segment<br>acknowledged | Response received                     |
| PUT,<br>server | First request<br>segment received | Last request segment<br>received     | Response sent                         |

Duration and average throughput are always computed on the side that receives the bulk data:

$$\begin{aligned} \text{GET: } & \frac{\text{bytes received by client}}{T_3 - T_2} \\ \text{PUT: } & \frac{\text{bytes received by server}}{T_2 - T_1} \end{aligned}$$

Time to first byte in the report is T2 at the client. Goodput counts payload delivered to the application; throughput also counts TCP headers and retransmitted segments.

Connection health checks from the TCP counters:

- At the client, SYN-received and established should coincide. A later SYN-received means a duplicate SYN+ACK: a small gap points at a misbehaving router, a gap longer than the retransmit time at lost ACKs.
- More than one SYN received is suspicious.
- Client SYN-sent to server established is a lower bound on connection setup time.
- Read a counter before its timestamp; a timestamp whose counter is zero raises an error.
- Do not compare last-packet timestamps between client and server on fast or long paths; use round-trip time.

## Traffic models

Application tests are frame-blasting or TCP flows shaped to resemble an application. These are the shapes the docs specify.

| Model                  | How the traffic is built                                                                                                                               | Defaults                                                                                             |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Voice                  | G.711 over RTP at a constant packet interval. 20 ms packetisation gives 50 packets per second of 160 bytes; 10 ms gives 100 per second of 80 bytes     | 20 ms                                                                                                |
| Gaming                 | UDP at a steady packet rate, with a set of frames whose lengths are drawn from a normal distribution, clamped to a minimum and maximum                 | Mean 110 bytes, deviation 20, min 22, max 1480, 20 distinct frames, 30 packets per second            |
| Video                  | A simulated streaming player described only by its parameters: segment size, segment duration, a play goal and a buffering goal. It runs until stopped | Segment size 2,000,000 (unit not stated), segment duration 2.5 s, play goal 5 s, buffering goal 60 s |
| Conference             | Three parallel frame-blasting streams: video, voice and screen share, each with its own rate and check                                                 | Set per stream                                                                                       |
| IMIX                   | A mix of frame sizes, each with a weight setting its share of the frames, optionally shuffled                                                          | Shuffled                                                                                             |
| Dynamic frame blasting | A UDP flow whose bitrate moves between a minimum and a maximum, updated once per scaling interval by a scaling rate                                    | 5 to 50 Mbit/s, 5% per step                                                                          |
| L4S frame blasting     | UDP frames carrying a chosen ECN codepoint: not-ECT, classic, L4S or CE                                                                                |                                                                                                      |
| HTTP                   | One request for a duration or a size, optionally capped at a maximum bitrate                                                                           | Unlimited                                                                                            |

Frame modifiers change a stream while it runs:

| Modifier          | Rule                                                                                                         | Defaults                              |
|-------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Growing size      | Send <code>iteration</code> frames at a size, then add <code>step</code> bytes, from minimum up to maximum   | 60 to 1514 bytes, step 1, iteration 1 |
| Random size       | Each frame gets a random size between minimum and maximum                                                    | 60 to 1514 bytes                      |
| Incremental field | A field of 1, 2, 4 or 8 bytes at a byte offset counts up by <code>step</code> between minimum and maximum    | Length 2, range 0 to 65535, step 1    |
| Random field      | The same field takes a random value in the range                                                             |                                       |
| Multiburst        | Send <code>burstsize</code> frames at the stream's rate, stay silent for <code>interburstgap</code> , repeat | 100 frames, 1 s gap                   |

The size modifier applies per frame. On a stream with frames A and B growing from 60, the order is A-60, B-60, A-61, B-61. Older API versions produced A-60, B-61, A-62, B-63. Growing size yields a sawtooth throughput graph; multiburst yields a square wave.

Auto-correction: when a modifier or tag changes a frame, the server can recompute the IP header checksum, IP length, layer-4 checksum and layer-4 length just before sending. These are off by default and cost server resources.

Protocol timers a port uses as a host:

| Protocol | Rule                                                                                                                                           |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------|
| DHCPv4   | Discover then Request, each with an initial timeout of 1 s and up to 5 retries; fixed timing or RFC 2131 backoff; renew at half the lease time |
| ARP      | Cache checked first; entries valid 120 s                                                                                                       |
| IPv6     | Link-local address derived from the MAC (modified EUI-64); gateway learned from router advertisements; source address chosen per RFC 6724      |

## Pass and fail logic

A verdict is a threshold test applied by an analyser attached to a flow. The run fails if any flow fails.

| Analyser             | Applies to        | Rule                                                                    | Default                   |
|----------------------|-------------------|-------------------------------------------------------------------------|---------------------------|
| Frame loss           | UDP flows, gaming | Loss % within the maximum                                               | 1.0%                      |
| Latency and loss     | UDP flows         | Loss within the maximum and average latency within the threshold        | 1.0%, 5 ms                |
| Latency CDF and loss | UDP flows         | Loss within the maximum and latency at the quantile below the threshold | 1.0%, quantile 99.9, 5 ms |
| Gaming               | Gaming flows      | 99th-percentile latency below the threshold                             | 5 ms                      |
| Voice                | Voice flows       | Mean opinion score at or above the minimum                              | 4                         |
| Video buffer         | Video flows       | Wait before playback starts within the maximum                          | 5 s                       |
| HTTP, L4S HTTP       | TCP flows         | None; always reported as "no analysis"                                  |                           |

The Low Latency test case adds `min_percentile` and `max_percentile` (defaults 10 and 90) as outlier boundaries for its latency analysis.

Standard-test verdicts:

**RFC 2544.** For every frame size, measured throughput must reach `expected_bitrate`.

**TR-398 Airtime Fairness.** For stations A and B with solo TCP maxima  $M$ , sent UDP at 0.75  $M$  for station 1 and 0.50  $M$  for its partner, and measured UDP rates  $U$ :

$$U_A > 0.45 \cdot M_A \quad \text{and} \quad U_B > 0.45 \cdot M_B \quad \text{and} \quad U_A + U_B > \text{expected throughput for the mode}$$

The two offered loads sum to more than the access point can carry, which forces it to choose who gets airtime. If station 1's solo test fails, the whole configuration is skipped.

**DOCSIS 4.0 ATP LL-04.40.** Two classic flows of 200-byte frames run back to back for 20 s each, the first at a higher rate than the second, both above the modem's configured limit. Loss % and latency are computed per 200 ms interval and compared with the ATP's test points, once without and once with a 40 s low-latency flow alongside.

**Scenario length.** With a maximum run time, every flow is cut at initial wait plus duration and the scenario stops at the maximum. Without one, the scenario runs as long as its longest flow, 10 s if none has a limit, and always waits for size-based TCP flows to finish.

## Timing machinery

### Scheduling

Every stream and every schedulable action (an HTTP request, a multicast join, a ping) has an initial time to wait. Starting a set of ports starts all those countdowns at the same instant, and each item fires when its own wait expires. That is how flows in a scenario are offset against each other. Items already running are ignored; stopped ones are scheduled again.

The API is synchronous and the server never pushes events, so a script polls: start, sleep, refresh results, repeat.

### Result sampling

The server keeps results in a fixed-length ring per object:

1. Every sampling interval (default 1 s) it closes one interval sample and one cumulative sample.
2. It keeps the newest few (default 6); each new sample pushes out the oldest.
3. **Refresh** copies whatever is in the ring to the client. The last sample is still open.
4. A client that refreshes less often than the ring's span (6 s by default) loses samples and gets gaps.

Refreshing many results in one batched call matters when comparing a sender with a receiver: separate round trips would read them at different moments. Scripts loop one extra second so frames on an interval boundary are counted.

### Clock synchronisation

| Pair                    | How clocks agree | Accuracy stated |
|-------------------------|------------------|-----------------|
| Two ports on one server | One clock        | Exact           |

| Pair                                    | How clocks agree                                                                                                                                           | Accuracy stated                           |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| Two servers with PTP (2100, 4100)       | Hardware clocks disciplined continuously over a dedicated cable                                                                                            | Within a couple of microseconds           |
| Two servers with NTP                    | Software discipline; on 2100 and 4100 only at startup                                                                                                      | Lower; keep latency to the NTP server low |
| Server and Endpoint                     | The Endpoint sets its clock from the Meeting Point once, at registration, over the same link as test traffic unless a separate management interface is set | Best effort; drifts during long tests     |
| Server and Golden Client (NUC with PTP) | PTP with NTP fallback                                                                                                                                      | Well below 1 ms on wired                  |

A PTP node listens first and becomes master only if it hears no other; a dedicated master is forced with `masterOnly`. PTP gives no wall-clock reference, so PTP-only systems drift from real time while staying aligned with each other.

## Endpoint run cycle

An Endpoint cannot be commanded live, so a test is shipped to it whole:

1. The Endpoint sends heartbeats to the Meeting Point and waits (Registered).
2. The controller locks it, uploads the scenario (Armed), and asks it to start. The reply is a start time slightly in the future.
3. Both sides wait for that time, then run (Running). By default the Endpoint sends no management traffic during the run.
4. Afterwards it reconnects on its next heartbeat and uploads results, which the Meeting Point relays. The framework waits up to 120 s for them.

With Heartbeat Mode (since 2.19.0) it keeps beating during the run, which allows remote cancel.

## Report size budget

The GUI limits a report to about 12,000 graphing-hours:

```
\text{duration in hours} \times \sum_{\text{flows}} w < 12000 \quad w = 1
\text{ (frame blasting)}, \ 5 \text{ (latency)}, \ 5 \text{ (TCP)}, \ 1 \text{ (out-of-sequence)}
```

Example: two TCP flows, one plain UDP flow and one UDP flow with latency weigh 17; five days is 120 h, so 2,040. Separately, scenarios with more than 100 flows store no over-time results at all.

## Named but never explained

I re-read the pages for each of these; the docs give parameters or a name and no method. The source of the Test Framework, which is an installable Python package, is where the answers would be.

| Topic                  | What the docs give                                          | What is missing                                                                                   |
|------------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Jitter                 | A value in every latency result                             | The definition or formula                                                                         |
| MOS for voice          | "Calculates the Mean Opinion Score"                         | The model and its inputs                                                                          |
| Video buffer           | Buffer size, play goal, maximum initial wait                | Fill and drain logic, stall rule, the exact pass criterion                                        |
| Dynamic frame blasting | Minimum and maximum bitrate, scaling interval, scaling rate | What triggers a step up or down, and what the 5% applies to                                       |
| Gaming packet sizes    | Normal distribution with mean, deviation, min, max          | Whether deviation is a standard deviation; how samples are clamped and interleaved                |
| IMIX                   | Length and weight per entry                                 | The selection method and the default mix                                                          |
| Quantile               | Latency at a quantile of the CDF                            | Bucket count and interpolation                                                                    |
| RFC 2544 reruns        | "Automated workarounds"                                     | How many reruns are allowed; units of <code>tolerated_frame_loss</code> and <code>accuracy</code> |
| RFC 2544 first phase   | "Doubling or increasing by 50%"                             | Which of the two the code uses                                                                    |
| Timestamp sanity check | That a check exists and depends on the tag format           | What it checks                                                                                    |
| DOCSIS ATP             | Procedure and interval                                      | Flow rates and pass thresholds                                                                    |
| TR-398                 | The 45% rule                                                | The expected-throughput table per mode                                                            |

| Topic               | What the docs give | What is missing                         |
|---------------------|--------------------|-----------------------------------------|
| Latency noise floor | A per-model chart  | The numbers, published only as an image |

Three things in this tab are my own additions, not from the docs: the estimate of how many RFC 2544 trials bisection needs, the 1 Gbit/s window-scale worked example, and the "Rule" column of the TCP diagnosis table, which states the reasoning behind symptoms and causes the docs list.